

Simple ReAct agents are effective for straightforward, one-shot tasks, but they can struggle when the task requires **long-term planning, multiple subtasks, context retention, and iterative reasoning**.

Deep Agents address this by adding capabilities such as:

- **Planning and task decomposition** – breaking a complex goal into smaller subtasks.
- **Context management** – maintaining relevant information across many steps.
- **Task tracking** – keeping track of completed and pending work.
- **Sub-agents** – delegating specialized subtasks when needed.
- **Re-planning** – adapting the plan based on intermediate results.
- **Long-running execution** – handling tasks that require many tool calls and iterations.

So, a simple way to remember it is:

ReAct tells an agent how to reason and act step-by-step.

Deep Agents give the agent the structure and capabilities needed to manage a complex task from start to finish.

Simple ReAct Agent

A ReAct (Reason + Act) agent typically follows a loop like:

User → Think → Tool → Observe → Think → Tool → Answer

It works well for relatively straightforward tasks such as:

“What is the population of Bangalore?”

But for a complex task like:

“Research the top 5 cloud providers, compare their AI offerings, pricing, enterprise adoption, and recent announcements, and give me a recommendation.”

a basic ReAct agent can struggle because it is essentially reacting one step at a time.

What does “ReAct reacts one step at a time” mean?

Suppose the user asks:

“Research whether our company should migrate from Informatica to Apache Beam/Python. Compare cost, performance, development effort, risks, and give me a recommendation.”

User asks question



Think: What should I do now?



Take one action



Observe result



Think again



Take another action



Observe result



...



Final answer

So the agent is essentially saying:

“Based on what I just observed, what should I do next?”

It isn't necessarily maintaining a strong **overall roadmap** such as:

My ultimate goal is to produce a migration recommendation.

To do that I need:

- ✓ Cost comparison
- ✓ Performance comparison
- ✓ Development effort
- ✓ Risks
- ✓ Migration complexity
- ✓ Recommendation

Deep Agent

A Deep Agent adds more structure around the agent's reasoning and execution.

A simplified view is:

User Request



Planning / Task Decomposition



Subtasks

- |— Research provider A
- |— Research provider B
- |— Research provider C
- |— Compare pricing
- |— Analyze enterprise adoption
- |— Review recent announcements



Store intermediate results / context



Review & synthesize
↓
Final Answer

The important additions are:

Capability	Simple ReAct	Deep Agent
Reason + act	✓	✓
Tool usage	✓	✓
Multi-step planning	Limited	✓
Task decomposition	Limited	✓
Long-running tasks	Weak	Stronger
Context management	Limited	Better
Intermediate results	Often implicit	Explicit/persistent

Sub-agents	Usually no	Can use them
Re-planning	Limited	
Complex research	Can struggle	Designed for it

Here,

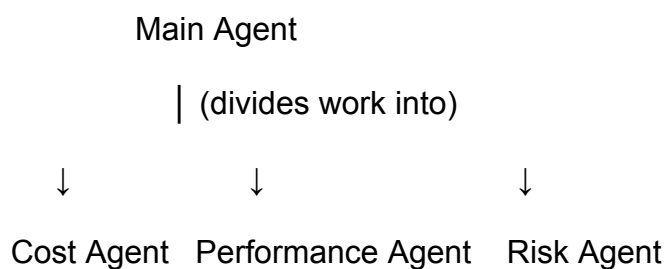
a) Re-planning means changing the plan when new information changes what needs to be done.

Initial plan: You are travelling from Bangalore to Mysore.

You discover: The highway is closed.

You change the route

b) sub-agent is another specialized agent that the main/orchestrator agent delegates a task to.



Instead of doing everything itself, it delegates into 3 subagents.

What Are Deep Agents?

Deep agents are a standalone library built on top of **LangGraph** that brings production-grade capabilities to LLM agents. Unlike simple agents that just loop between “think, act, observe,” deep agents come with built-in infrastructure for handling real-world complexity.

Here is what makes them different from a basic ReAct agent:

Planning -- Deep agents automatically break down complex tasks into subtasks using a built-in `write_todos` tool. Before diving into execution, they create a plan of attack.

File System for Context Management -- LLMs have limited context windows. Deep agents solve this by offloading large intermediate results (like search results or document content) into a virtual file system using `write_file` and `read_file` tools. This means they can handle tasks that produce far more data than a context window could hold.

LLMs have a **limited context window**, so they cannot keep all the information from a large, multi-step task in their active context at once. Deep Agents address this by using a **virtual file system as external storage**. Large intermediate results, such as search results, document contents, research notes, or analysis, can be saved using `write_file` instead of keeping everything in the LLM's context.

When the agent needs the information later, it can use `read_file` to retrieve the relevant content and continue reasoning. This allows the agent to work with **far more information than can fit in its context window**. A simple way to think about it is: **the LLM context is the agent's working desk, while the virtual file system is its filing cabinet**—it keeps only what it currently needs on the desk and stores the rest for later.

The Architecture Behind Deep Agents

User Input --> Planner (`write_todos`) --> Execute Step 1 --> File System (`write_file`)
--> Execute Step 2 --> Subagent (if needed) --> File System (`read_file`) -->
Synthesize --> Final Response

Memory in Langchain and Langgraph

Short-Term Memory

Short-term memory tracks active conversations within a single session or thread.

Scope: Limited to one specific thread ID.

Mechanism:

- Managed as part of the agent's graph state and persisted using a checkpointer (such as InMemorySaver, SQLite, or Postgres).
- It uses Checkpointers to save the state of the graph at every step.
- `from langgraph.checkpoint.memory import MemorySaver`
- `langgraph_checkpoint_postgres` or `langgraph_checkpoint_sqlite`

Long-Term Memory

Long-term memory retains data across multiple separate conversations and sessions.

Scope: Shared across different threads and scoped to custom namespaces (like user IDs or organization tags).

Mechanism:

- Built on LangGraph Stores, which save data as JSON documents managed by a namespace, key, and value.
- It stores profile information, user preferences, or facts (e.g., "User prefers Python over Java").
- It uses Stores to read and write data under specific namespaces (like `user_id`) across any execution thread.
- `from langgraph.store.memory import InMemoryStore`
- `langgraph.store.postgres`

Example

```
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
```

```

from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import MemorySaver # Short-term
from langgraph.store.memory import InMemoryStore # Long-term

# 1. Define State (Short-term context)
class State(TypedDict):
    messages: Annotated[list, add_messages]

# 2. Define Node that uses both memories
def chat_node(state: State, config: dict, store: InMemoryStore):
    # Retrieve user_id from config to access long-term memory
    user_id = config["configurable"].get("user_id")

    # Fetch long-term memory profile
    namespace = ("profiles", user_id)
    user_profile = store.get(namespace, "preferences")

    # Business logic representation
    print(f"--- Long-Term Memory Retrieved for {user_id}: {user_profile.value if user_profile else 'None'} ---")
    print(f"--- Short-Term Chat History (Current Thread): {len(state['messages'])} messages ---")

    # Update long-term memory if needed (simulated update)
    if not user_profile:
        store.put(namespace, "preferences", {"language": "Python"})

    return {"messages": [{"role": "assistant", "content": "Got it!"}]}

# 3. Compile Graph with Short-term memory (checkpointer)
builder = StateGraph(State)
builder.add_node("chat", chat_node)
builder.add_edge(START, "chat")
builder.add_edge("chat", END)

# Short-term memory allocation
checkpointer = MemorySaver()

# Long-term memory allocation
long_term_store = InMemoryStore()

```



```
# Compile everything together
graph = builder.compile(checkpointer=checkpointer, store=long_term_store)

# Thread 1, User 'alice'
config_1 = {"configurable": {"thread_id": "thread_abc", "user_id": "alice"}}
graph.invoke({"messages": [{"role": "user", "content": "Hi, I like Python."}]}, config_1)

# Thread 2, Same User 'alice' (Short-term reset, Long-term persists)
config_2 = {"configurable": {"thread_id": "thread_xyz", "user_id": "alice"}}
graph.invoke({"messages": [{"role": "user", "content": "What language do I like?"]}],
config_2)
```

Middleware in Langchain and Langgraph

In LangGraph, middleware is defined as a composable set of lifecycle hooks that intercept data flowing through an agent's execution loop.

Middleware is an invisible "filter" or "checkpoint" that sits right in the middle of your program to inspect, change, or stop data before it moves to the next step.

Instead of writing safety or logging code directly inside your main application logic, you put it in middleware so it handles those tasks quietly in the background.

The Real-World Analogy: Airport Security

Think of an AI Agent as a **passenger** flying out of an airport:

- **Without Middleware:** The passenger walks straight from the street right onto the airplane. There is no ID check, no luggage scan, and no ticket verification. If you want security, you have to manually build a security desk inside the airplane cabin.
- **With Middleware:** Airport security acts as the middleware. Before the passenger (the prompt/data) can ever reach the plane (the LLM model), they must pass through checkpoints:
 1. **Checkpoint 1 (Luggage Scan):** Checks for dangerous items (filters out private data like credit card numbers or PII).
 2. **Checkpoint 2 (Ticket Counter):** Checks if you can afford the flight (ensures you haven't run out of API money or token budget).

The Middleware Architecture

The different types of Middleware include:

before_agent / after_agent: Runs at the entry and exit points of the entire agent execution. Runs when the entire workflow starts and ends. Ideal for loading initial memory state or generating final analytics dashboards.

before_model / after_model: Runs directly before data reaches the LLM and immediately after it returns. Runs right before messages hit the LLM and right after the LLM generates a response. Ideal for input/output guardrails.

modify_model_request: Intercepts and alters the payload going to the LLM (e.g., adding prompt caching tags). Allows you to change the actual structure of the payload (e.g., dynamically adding prompt caching tokens or limiting which tools the LLM can see for this specific turn).

SummarizationMiddleware: automatically summarises dialogue histories when specific token or message counts are reached, maximizing context window efficiency.

Guardrails & Costs: ModelCallLimitMiddleware and ToolCallLimitMiddleware prevent rogue autonomous agents from infinite loops, enforcing hard stops or graceful error handling after a set number of runs.

HumanInTheLoopMiddleware adds declarative interrupts to sensitive tool executions (like send_email), pausing execution until a human operator grants permission.

PIIMiddleware- if you want a guardrail that blocks the LLM from leaking credit card numbers, you implement a PIIMiddleware. The middleware intercepts the response on its way out and enforces the PII-redaction guardrail.

Are Middleware and Guardrails the Same?

No, they are not the same. They represent two completely different concepts, though they heavily overlap in how they are implemented.

The Difference: Guardrails define what you want to enforce (the security or safety policy). Middleware is how you inject that rule cleanly into your software stack without hardcoding it everywhere.

Middleware as the vehicle to deploy guardrails.

Generic code for implementing middleware using langgraph

It can be done using 2 methods:

The Native Method (Using `create_agent`): The simplest and most standard approach in LangGraph. You write modular functions using lifecycle hooks (`@before_model`, `@after_model`, etc.) and supply them to the agent graph builder.

The Manual Method (Pure `StateGraph` nodes): Used when building a completely customized graph topology without using prebuilt agent factories.

Pattern 1: The Native Method (`create_agent`) This is the recommended strategy. It leverages decorators to inject code directly into the agent's execution loop, cleanly decoupling background tasks from core application logic.

```
from typing import Any
from typing_extensions import NotRequired
from langchain.agents import create_agent, AgentState
from langchain.agents.middleware import before_model, after_model
from langgraph.runtime import Runtime
from langchain_openai import ChatOpenAI

# 1. (Optional) Extend the default AgentState to track custom metadata
class CustomState(AgentState):
    request_id: NotRequired[str]
    processing_time: NotRequired[float]

# 2. Generic Input Middleware (Runs BEFORE the LLM is called)
# Setting can_jump_to=["end"] allows you to short-circuit the graph under custom
conditions
@before_model(state_schema=CustomState, can_jump_to=["end"])
def generic_input_middleware(state: CustomState, runtime: Runtime) -> dict[str, Any] |
None:
    print("--- Executing Input Middleware ---")

    # Process or mutate input data here
    # Example condition to halt the graph early:
    if len(state["messages"]) > 20:
        return {"jump_to": "end"} # Early termination
```

```
return None # Proceed to the LLM normally
```

```
# 3. Generic Output Middleware (Runs AFTER the LLM yields a response)
```

```
@after_model(state_schema=CustomState)
```

```
def generic_output_middleware(state: CustomState, runtime: Runtime) -> dict[str, Any] |
```

```
None:
```

```
    print("--- Executing Output Middleware ---")
```

```
    # Track metrics, log response text, or parse outputs here
```

```
    # Return updates to merge directly into the graph's State
```

```
    return {"processing_time": 0.45}
```

```
# 4. Initialize model and compile the complete graph
```

```
llm = ChatOpenAI(model="gpt-4o-mini")
```

```
# LangGraph implicitly compiles these middleware handlers directly into graph nodes
```

```
agent_graph = create_agent(
```

```
    model=llm,
```

```
    tools=[],
```

```
    state_schema=CustomState,
```

```
    middleware=[generic_input_middleware, generic_output_middleware]
```

```
)
```

Pattern 2: Manual Node Insertion (Pure StateGraph) If you choose not to use `create_agent` and are engineering a deep, multi-agent custom graph with `StateGraph(MessagesState)`, there is no native `.add_middleware()` function. Instead, middleware is implemented explicitly by dropping interceptor functions as dedicated nodes sequentially preceding or succeeding your main LLM execution step.

```
from typing import TypedDict
```

```
from langgraph.graph import StateGraph, START, END
```

```
from langchain_core.messages import BaseMessage
```

```
# Define graph state
```

```
class GraphState(TypedDict):
```

```
    messages: list[BaseMessage]
```

```
    metadata: dict
```

```

# Define the model node
def model_node(state: GraphState):
    print("-> Step 2: Main LLM Node Executing")
    # Call your LLM here...
    return {"messages": state["messages"]}

# --- MIDDLEWARE CODE AS GRAPH NODES ---

def custom_before_model_middleware(state: GraphState):
    print("-> Step 1: Middleware Intercepting (Before Model)")
    # Edit the prompt, prune history, or run guardrails here
    return state

def custom_after_model_middleware(state: GraphState):
    print("-> Step 3: Middleware Intercepting (After Model)")
    # Scan outputs, update analytics trackers, evaluate metrics here
    return state

# --- GRAPH ASSEMBLY ---

workflow = StateGraph(GraphState)

# Register all steps as structural nodes
workflow.add_node("input_middleware", custom_before_model_middleware)
workflow.add_node("llm_model", model_node)
workflow.add_node("output_middleware", custom_after_model_middleware)

# Construct sequential control flow edges
workflow.add_edge(START, "input_middleware")
workflow.add_edge("input_middleware", "llm_model")
workflow.add_edge("llm_model", "output_middleware")
workflow.add_edge("output_middleware", END)

# Compile into a runnable graph
custom_compiled_graph = workflow.compile()

```

1. What is Runnable Interface in Langchain?

Runnable Interface is a standard way to run and connect langchain components. A runnable can be composed with other runnable. For example the Prompt, Model and Parser can be joined into one flow. Common methods include invoke, batch and stream.

```
chain = prompt | model | parser
```

2. Tool Calling in Langchain

You give the model a set of tools. The model decides whether it needs a tool and, if so, generates a structured request to call it.

3. How to reduce hallucination in a LangChain RAG system

1. Improve retrieval first

A common mistake is to focus only on the prompt.

If the retriever gives the LLM bad context, the LLM has little chance of producing a correct answer.

For example:

```
retriever = vectorstore.as_retriever(  
    search_kwargs={"k": 5}  
)
```

Experiment with:

- k
- chunk size
- chunk overlap
- embedding model
- similarity threshold
- metadata filtering

For example, if your chunks are too large- the model may struggle to identify the relevant information.

2. Use a strong grounding prompt

Tell the model explicitly what it is allowed to use.

For example:

```
prompt = ""
```

You are a question-answering assistant.

Answer the question using ONLY the provided context.

If the answer cannot be found in the context, say:

"I don't know based on the provided documents."

Do not use outside knowledge.

Do not make up facts.

Context:

{context}

Question:

{question}

Answer:

""

This is much safer than: "Answer the user's question."

3. Don't blindly trust retrieved documents

You can add a relevance threshold.

Question



Retriever



10 documents



Relevance filtering/reranking

↓
2-3 highly relevant documents
↓
LLM

This is often better than simply increasing k.

A reranker can be particularly useful when vector similarity retrieves documents that are semantically similar but don't actually answer the question.

4. Consider query rewriting

The user's question isn't always a good retrieval query.

For example: "What about its refund policy?"- The retriever doesn't know what "its" refers to.

You can first rewrite: "What is Apple's refund policy?"- and then retrieve documents.

5. Don't put too much context into the LLM

More documents ≠ better RAG.

For example: k = 50 -might actually make things worse because the model receives a huge amount of irrelevant information.

Prefer-
retrieve many
↓
rerank/filter
↓
send only the best few

6. Make the answer cite its sources

You can ask the model to provide citations:

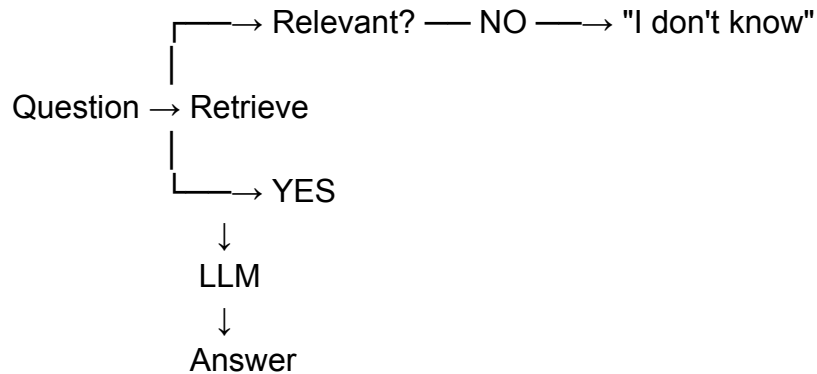
Answer the question using only the context.

For every factual statement, provide the source document/page that supports it.

If there is insufficient evidence, say you don't know.

7. Add an "answerability" check

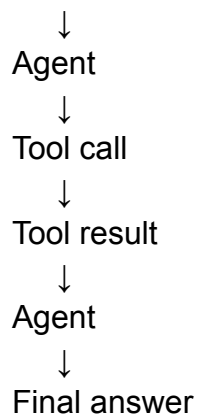
A stronger RAG architecture is:



4. What is timetravel in Langgraph

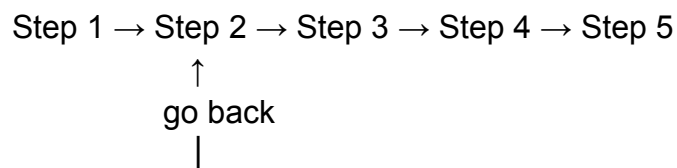
In Langgraph, timetravel is going back to an earlier state/checkpoint of an agent's execution and continuing from there.

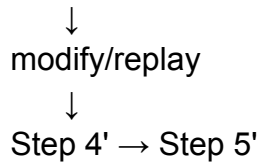
User question



Suppose the agent made a mistake at step 4.

With time travel, you can go back to an earlier checkpoint:





It's particularly useful for debugging and agent workflows.

4. How to reduce large language model (LLM) token usage and cost

To reduce large language model (LLM) token usage and cost, use a combination of model routing, prompt caching, and context optimization:

Optimize Input Tokens

Implement Model Routing: Route simple tasks (like classification or basic extraction) to smaller, cheaper models (such as GPT-4o mini) and reserve flagship models only for complex reasoning.

Use Retrieval-Augmented Generation (RAG): Instead of stuffing entire documents into a prompt, use RAG to retrieve only the specific text chunks needed to answer a query.

Enable Prompt Caching (supported by Anthropic and OpenAI): Prompt Caching allows an LLM provider to store a long, frequently used prompt in memory. When you send a new request that shares that exact same long text, the model does not have to re-process it from scratch. You only pay a heavily discounted rate for the cached portion.

Think of it like a library textbook: instead of the model buying a new copy of the textbook for every single question you ask, it keeps the book open on the desk and only charges you a small fee to look at the pages it already read.

Example Scenario Imagine you are building a legal AI assistant that helps lawyers analyze a specific 50-page contract (roughly 30,000 tokens).

Without Caching: Every time the lawyer asks a question (e.g., "What is the termination clause?" followed by "Is there a non-compete?"), you send the entire 50-page contract again. You pay the full price for 30,000 input tokens on every single turn.

With Caching: The first question processes the full contract at standard price. For the second, third, and twentieth questions, the AI provider recognizes the contract is already cached. You are only billed the cached input token rate (which is up to 50% to 90% cheaper on platforms like Anthropic or OpenAI) for that contract block.

Implementing Prompt Caching varies depending on your provider.

OpenAI optimizes automatically for prompts above 1,024 tokens. Your only job is keeping your prompt structure identical.

Anthropic (Claude) and Azure OpenAI allow explicit control, where you declare precise "breakpoints" in your code to pin static context in memory.

1. Anthropic Claude (Python):

Anthropic requires you to pass a `cache_control` block. You can flag the large background text block (like a system prompt, document, or tool definition) as `ephemeral`.

In platforms like the Claude Platform Docs, setting `cache_control={"type": "ephemeral"}` tells the API to temporarily cache prompt blocks (with a default 5-minute lifetime) to speed up response times and reduce token costs on repeated calls.

```
import os
from anthropic import Anthropic

client = Anthropic(api_key=os.environ.get("ANTHROPIC_API_KEY"))

# 1. Load your large static text (e.g., a massive legal document or code base)
with open("massive_contract.txt", "r") as f:
    large_context_document = f.read()

# 2. Call the API using an explicit cache breakpoint
response = client.messages.create(
    model="claude-3-5-sonnet-latest",
    max_tokens=1024,
    system=[
        {
            "type": "text",
```

```

        "text": "You are an expert legal assistant. Analyze the document provided
below.",
        # This ensures the core system prompt is cached if it meets token minimums
        "cache_control": {"type": "ephemeral"}
    }
],
messages=[
    {
        "role": "user",
        "content": [
            {
                "type": "text",
                "text": large_context_document,
                # Crucial: Pin the end of this massive static document in the cache
                "cache_control": {"type": "ephemeral"}
            },
            {
                "type": "text",
                "text": "What is the expiration date specified in this contract?"
            }
        ]
    }
]
)

```

```

# 3. Monitor your token usage to verify the cache hit
print(f"Cached tokens read: {response.usage.cache_read_input_tokens}")
print(f"Cache write creation tokens: {response.usage.cache_creation_input_tokens}")

```

2. Automatic Caching: OpenAI (Python)

OpenAI automatically caches text prefixes longer than **1,024 tokens** **Lifespan: Caches are typically held for 5 to 10 minutes of inactivity and are fully cleared within 1 hour of their last use.** . You do not need to add unique parameters; you just need to ensure the static data is positioned first in the prompt array.

Control Output and System Costs

Constrain Output Length: Set explicit maximum output token limits and use structured outputs (like JSON) to keep answers concise, since output tokens cost significantly more than input tokens.

Replace LLM Calls with Code: Use traditional regex, classic NLP, or deterministic code for logic steps that do not require generative AI.

